

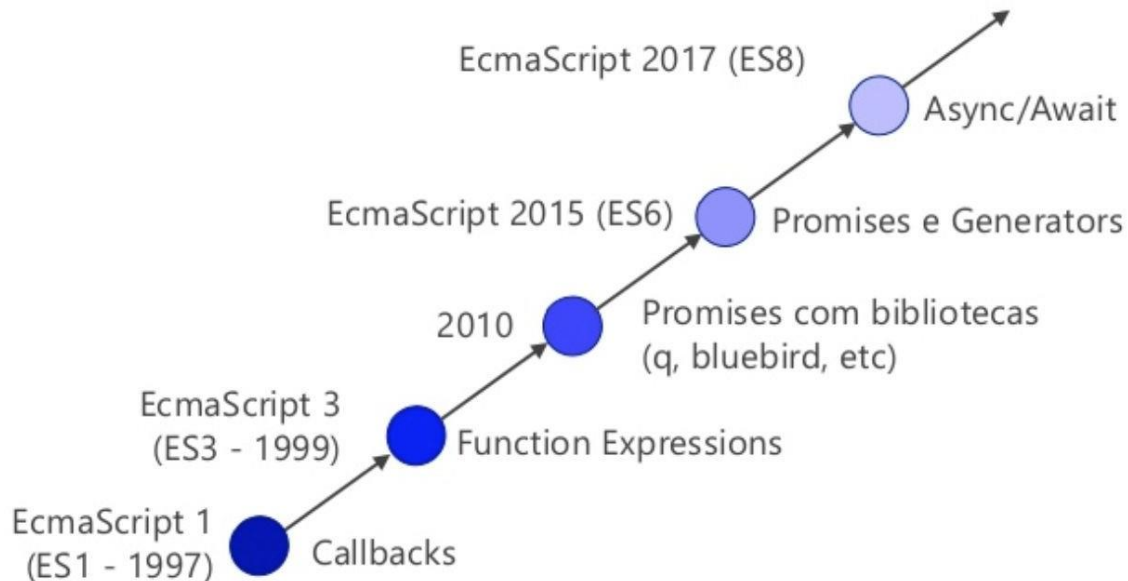
JavaScript

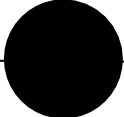
Funções Map, Filter e Reduce

Sumário

- ◉ Funções JavaScript – Array
 - ◉ Map
 - ◉ Filter
 - ◉ Reduce
- ◉ Função e Parâmetros de Função
- ◉ Default Parameters
- ◉ Destructuring Assignment

Evolução





Funções JavaScript – Array

Função Map

- É utilizada para traduzir/mapear todos os elementos de um *array* para algum outro conjunto de valores. Ela percorre o *array* da esquerda para a direita invocando uma função de retorno (com parâmetros) para cada elemento percorrido.
- Para cada chamada de retorno, o valor devolvido se torna elemento do novo array. Após todos os elementos serem percorridos, o **map()** retorna o novo array com todos os elementos “traduzidos”.

Função Map

Parâmetro	Significado
elem	Valor do elemento
index	Índice em cada iteração, da esquerda para a direita
<i>array</i>	<i>Array</i> original invocando o método
thisArg	(opcional) Objeto que será referenciado como this no callback

Protótipo

```
array.map( function( elem, index, array ) {  
    ...  
}, thisArg );
```



Função Map

```
// Exemplo utilizando Array Simples
var metros = [ 1, 5, 3, 2 ];

var milimetros = metros.map( function( elem ) {
    return elem * 1000;
});

for(var a=0; a < milimetros.length; a++) {
    console.log(milimetros[a] + ' mm');
}

// Exemplo utilizando Array Simples - ES6
var metros = [ 1, 5, 3, 2 ];

// Função “milimetros” utilizando Sintaxe ES6
var milimetros = metros.map( elem => elem * 1000 );

for(var a=0; a<milimetros.length; a++) {
    console.log(milimetros[a] + ' mm');
}
```

```
// Exemplo utilizando Array de Objetos
var cidades = [
    { nome : 'Xanxerê', temperatura : 32 },
    { nome : 'Chapecó', temperatura : 18 },
    { nome : 'Florianópolis', temperatura : 35 },
    { nome : 'São Jose', temperatura : 30 }
];
// Função “kelvin” utilizando Sintaxe ES5
var kelvin = cidades.map( function( elem ) {
    return elem.temperatura + 273;
});

for(var a=0; a<kelvin.length; a++) {
    console.log(kelvin[a] + '° K');
}

// Função “kelvin” utilizando Sintaxe ES6
var kelvin = cidades.map( ( elem ) => elem.temperatura + 273 );

for(var a=0; a<kelvin.length; a++) {
    console.log(kelvin[a] + '° K');
}
```


- É utilizada para remover elementos indesejados com base em alguma(s) condição(ões). Ela percorre o *array* da esquerda para a direita invocando uma função de retorno para cada elemento percorrido
- O valor retornado é *booleano*, e indica se o elemento percorrido será mantido no novo array ou descartado. Após todos os elementos serem percorridos, a função **filter()** retorna o novo array com todos os elementos que retornaram verdadeiro

Função Filter

Parâmetro	Significado
elem	Valor do elemento
index	Índice em cada iteração, da esquerda para a direita
array	Array original invocando o método
thisArg	(opcional) Objeto que será referenciado como <i>this</i> no <i>callback</i>

Protótipo

```
array.filter( function( elem, index, array ) {  
    ...  
}, thisArg );
```

Função Filter

Exemplo de Codificação

```
// Exemplo utilizando Array Simples
```

```
function aprovado(arr) {  
    return arr >= 7;  
}
```

```
var notas = [10, 5, 6, 3, 9];  
var ap = notas.filter(aprovado);
```

```
for(var a=0; a<ap.length; a++) {  
    console.log(ap[a]);  
}
```



Função Filter

// Exemplo utilizando Array de Objetos

```
function aprovado(obj) {  
    return obj.nota >= 7;  
}  
  
var alunos = [  
    {nome : 'Maria', nota : 10},  
    {nome : 'Rodrigo', nota : 5},  
    {nome : 'Gabriela', nota : 3},  
    {nome : 'Bruno', nota : 7}  
];  
  
var ap = alunos.filter(aprovado);  
  
for(var a=0; a < ap.length; a++) {  
    console.log(ap[a].nome);  
}
```

- ◉ É utilizada para computar um valor cumulativo ou concatenado, tendo como base todos os elementos de um *array*. Ela o percorre da esquerda para a direita invocando uma função de retorno para cada elemento.
- ◉ O cálculo retornado é o valor acumulado e passado a cada nova chamada da função (*callback*). Após todos os elementos serem percorridos, a função *reduce()* retorna o valor acumulado/concatenado



Função Reduce

- É utilizada para computar um valor cumulativo ou concatenado, tendo como base todos os elementos de um **array**. Ela o percorre da esquerda para a direita invocando uma função de retorno para cada elemento.
- O cálculo retornado é o valor acumulado e passado a cada nova chamada da função (**callback**). Após todos os elementos serem percorridos, a função **reduce()** retorna o valor acumulado/concatenado.

Protótipo

```
array.reduce ( function ( prevVal, elem, index, array )  
  {  
    ...  
  }, initialValue );
```

Função Reduce

Parâmetro	Significado
preVal	Valor acumulado e retornado em cada iteração
elem	Valor do elemento
index	Índice em cada iteração, da esquerda para a direita
<i>Array</i>	<i>Array</i> original invocando o método
initalValue	(opcional) Objeto usado como primeiro argumento na primeira iteração.

Protótipo

```
array.reduce ( function ( prevVal, elem, index, array )  
  {  
    ...  
  }, initialValue );
```

Função Reduce

```
// Exemplo Utilizando Simples
```

```
var titulos = [4, 2, 1, 5];
```

```
// Função “soma” utilizando Sintaxe ES5
```

```
var soma = titulos.reduce( function( prevVal, elem ) {  
    return prevVal + elem;
```

```
}, 0);
```

```
console.log(soma);
```

```
// Função “soma” utilizando Sintaxe ES6
```

```
var soma = titulos.reduce( ( prevVal, elem ) => prevVal + elem, 0);
```


Função Reduce

```
// Exemplo Utilizando Array de Objetos
```

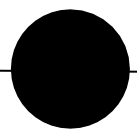
```
var selecoes = [  
  { pais : 'Alemanha', titulos : 4 },  
  { pais : 'França', titulos : 2 },  
  { pais : 'Espanha', titulos : 1 },  
  { pais : 'Brasil', titulos : 5 }  
];
```

```
// Função “soma” utilizando Sintaxe ES5
```

```
var soma = selecoes.reduce( function( prevVal, elem ) {  
  return prevVal + elem.titulos;  
}, 0);  
console.log(soma);
```

```
// Função “soma” utilizando Sintaxe ES6
```

```
var soma = selecoes.reduce( ( prevVal, elem ) => prevVal + elem.titulos, 0);  
console.log(soma);
```



Função e Parâmetros de Função



Função e Parâmetros de Função

- **Default Parameters:** possibilita ao desenvolvedor definir valores *default* para os parâmetros das funções.

```
// Exemplo - Padrão ES6
var multiplicar = function(x=1, y=1) {
  return x * y;
}

// Exemplo - Parâmetro default + Arrow Function
var multiplicar = (x=1, y=1) => x * y;

console.log('Resultado: ' + multiplicar());
console.log('Resultado: ' + multiplicar(5));
console.log('Resultado: ' + multiplicar(5, 3));
```

Função e Parâmetros de Função

- ◉ **Destructuring Assignment (atribuição via desestruturação):** possibilita extrair dados de arrays ou objetos em variáveis distintas.

Exemplo de Codificação

```
// Destructuring Assignment - Array  
const [a, b] = [10, 20];  
console.log(a + " " + b);
```

Observe que ao declararmos as constantes “a e b” atribuímos os valores “10 e 20” vindos do *array*.

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment

Função e Parâmetros de Função

```
// Destructuring Assignment - Array + Rest  
const [a, b, ...rest] = [10, 20, 30, 40, 50];  
console.log(a + " " + b + " " + rest);
```

Observe que ao declararmos as constantes “a, b e rest”, “a e b” recebem os valores “10 e 20” e o “rest” recebe todos os outros valores (“30, 40, 50”) do *array*.

```
// Destructuring Assignment - Objeto  
const aluno = { nome : 'Matheus', idade : 17 };  
const {nome, idade} = aluno;  
  
console.log(nome + " / " + idade + " anos");
```

Observe que ao declararmos as constantes “nome, idade”, elas recebem, respectivamente, os valores ‘Matheus’ e ‘17’ do array de objeto. Essa abordagem é permitida desde que as constantes (`const {nome, idade}`) tenham o mesmo nome do identificador do atributo do objeto, nesse caso “nome” e “idade”.

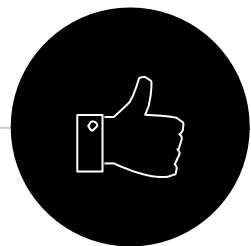
Exercícios Práticos 02

Conceitos Básicos JavaScript – Funções

Referências

[1] W3C School. JavaScript Tutorial. Disponível em:
<http://www.w3schools.com/js/>

[2] W3C School. React Tutorial. Disponível em:
<https://www.w3schools.com/react/>



Obrigado!

Dúvidas ?

Você pode me encontrar em:

- jackson.meires@ifsc.edu.br



@jacksonm